

«СОГЛАСОВАНО»
Зав. структурным
подразделением
специальности 09.02.07
«Информационные системы
и программирование»
Овчинникова С.Н.

«__» _____ 2026 г.

ЗАДАНИЕ

на курсовую работу

ПМ 01 «Разработка модулей программного обеспечения для компьютерных систем»

МДК 01.01 «Разработка программных модулей»

студенту _____
специальности 09.02.07 «Информационные системы и программирование»
Наименование темы _____

Техническое задание

Разработать настольное программное приложение для автоматизации обработки данных БД директором пункта ремонта электроприборов, с использованием СУБД MS SQL Server 2022, среды разработки Microsoft Visual Studio 2026, технологии доступа к данным ADO.NET и под управлением операционных систем семейства Windows.

Требования к приложению

1. требования к функциональным характеристикам

1.1 требования к серверной части

- реализация SQL- скрипта для создания структуры хранения данных
- реализация SQL- скрипта для создания набора тестовых данных

1.2 требования к клиентской части

- разработка интерфейсных форм пользователя приложения БД для ввода и редактирования данных
- разработка интерфейсных форм пользователя приложения БД для сортировки, поиска и фильтрации данных
- разработка бизнес-ограничений приложения БД на ввод имени, даты, номера телефона
- разработка отчетных форм приложения БД для оформления билетов на выбранный рейс

2. требования к надежности и безопасности

2.1 реализация защитного кодирования

- контроль входной и выходной информации приложения БД
- перехват ошибок при реализации логики приложения БД

2.2 реализация политики безопасности

- обеспечение целостности данных БД
- создание авторизованного доступа к ресурсам и функциям приложения БД

3. требования к информативности

- реализация справочной системы приложения БД
- разработка программной документации в виде пояснительной записки к курсовой работе

Содержание пояснительной записки курсовой работы

Титульный лист

Лист задания

Содержание

Введение

1. Постановка задачи

1.1. Содержательная постановка задачи

1.2. Анализ предметной области

2. Разработка приложения

2.1. Разработка серверной части приложения

2.1.1. Разработка логической модели данных

2.1.2. Разработка физической модели данных

2.1.3. Листинг SQL-операторов

2.2. Разработка клиентской части приложения

2.2.1. Разработка структуры клиентской части приложения

2.2.2. Разработка пользовательского интерфейса приложения

2.2.3. Разработка диалога авторизации приложения

2.2.4. Разработка программных модулей приложения

2.2.5. Разработка печатных и отчетных форм приложения

2.2.6. Разработка наборов тестов

2.2.7. Результаты работы приложения

2.3. Разработка руководства пользователя приложения

ЗАКЛЮЧЕНИЕ

Список используемых источников

Приложение 1 Входные документы

Приложение 2 Интеллектуальная карта

Приложение 3 Логическая модель данных

Приложение 4 Физическая модель данных

Приложение 5 Структурная схема приложения

Приложение 6 Листинг сценария создания таблиц

Приложение 7 Печатная форма приложения

Требование к защите курсовой работы

- индивидуальная защита работы с использованием электронной презентации с демонстрацией приложения БД

Дата выдачи задания _____

Дата окончания работы _____

Руководитель работы _____

Задание рассмотрено на заседании ПЦК профессиональных дисциплин специальности 09.02.07 «Информационные системы и программирование» Протокол №2 от 12.09.2025

председатель ПЦК _____

ВВЕДЕНИЕ

В настоящее время большинство компаний используют различные программные решения для обработки данных и их документирования. Рынок продажи железнодорожных билетов предъявляет высокие требования к скорости обработки информации, прозрачности учёта и качеству документооборота. Эффективное управление базой маршрутов и поездов, контроль бронирований и продаж, отслеживание платежей и формирование отчётной документации являются ключевыми факторами успешной работы билетного оператора.

В данной работе основной целью является создание программного решения для автоматизации деятельности сервиса по продаже железнодорожных билетов, обеспечивающего учёт железнодорожных рейсов, проданных билетов и финансовых операций.

Задачами являются:

- исследование специфики данной предметной области;
- проектирование базы данных информационной системы;
- разработка клиентского приложения для работы с базой данных;
- создание руководства пользователя приложения.

Объект изучения: Ж\Д Вокзал.

Предмет изучения: Ведение учёта продаж билетов каждым сотрудником и процесс документирования операций в системе продажи железнодорожных билетов.

1. ПОСТАНОВКА ЗАДАЧИ

1.1. Содержательная постановка задачи

Областью применения клиентского приложения является продажа железнодорожных билетов.

Продажа осуществляется на основании билета. Он формируется пассажиром совместно с кассиром/сотрудником билетной кассы или самостоятельно через терминал/веб-приложение. Оплата билета может выполняться несколькими способами, включая наличный и безналичный расчёт.

Билет проходит структуру статусов от бронирования до завершения поездки и содержит следующие параметры:

- номер и дата билета (номер билета и дата отправления);
- сумма и способ оплаты;
- данные пассажира (ФИО, документ, удостоверяющий личность);

данные о маршруте (станция отправления, станция прибытия, дата и время поездки, номер поезда и вагона, место).

После оформления билета сотрудник или терминал предоставляет посадочный документ и чек об оплате пассажиру.

Система имеет перечень доступных маршрутов и поездов (с расписанием, классами обслуживания и наличием мест), а также список перевозчиков, базу сотрудников и постоянных пассажиров (с историей поездок и накопленными скидками).

1.2. Анализ предметной области

По результатам анализа предметной области была составлена интеллектуальная карта (концептуальная модель), представленная в приложении 2.

2. РАЗРАБОТКА ПРИЛОЖЕНИЯ

2.1. Разработка серверной части приложения

База данных должна содержать следующие основные сущности: билеты, маршруты (поездки), пассажиры, сотрудники, пользователи, роли (должности).

А также вспомогательные сущности: статусы билетов, станции, города, типы вагонов, вагоны, места, детские билеты, паспортные данные.

2.1.1. Разработка логической модели данных

Цель логического проектирования – развить концептуальное представление БД с учетом принимаемой модели БД.

Логическая модель данных (ER-диаграмма) представлена в приложении 3.

Описание сущностей:

1. Должность (Post) — роли/должности пользователей информационной системы (например, «Директор», «Менеджер», «Покупатель»). Содержит наименование должности.

2. Сотрудник (User) — штатные сотрудники системы продажи билетов (кассиры, менеджеры). Содержит личные данные (имя, фамилию, отчество), логин, пароль для входа в систему, фотографию и ссылку на должность (PostId).

3. Пассажир (User) — физические лица, приобретающие билеты. В текущей реализации пассажиры также хранятся в таблице User (роль «Покупатель»). Содержит ФИО, логин, пароль (для возможной авторизации в личном кабинете) и паспортные данные (вынесены в отдельную таблицу).

4. Паспорт (Passports) — паспортные данные пассажира. Содержит серию, номер, кем выдан, дату выдачи и ссылку на пассажира (PassengerId).

5. Город (City) — географические населённые пункты, где расположены железнодорожные станции. Содержит название города.

6. Станция (RailwayStation) — железнодорожные станции отправления и прибытия. Содержит название станции и ссылку на город (CityId).

7. Поезд (Train) — подвижной состав. Содержит наименование поезда (например, «Классный поезд»).

8. Тип вагона (TypeCarriage) — классификация вагонов (например, «Плацкарт», «Купе», «Люкс»).

9. Вагон (Van) — конкретные вагоны, входящие в состав поездов. Содержит название вагона и ссылку на тип вагона (TypeCarId). Связь с поездом реализована через таблицу TrainVan.

10. Место (Place) — посадочные места в вагонах (например, «a1», «b2», «v3»). Содержит номер/обозначение места. Связь с вагоном реализована через таблицу VanPlace.

11. Маршрут / Поездка (Trip) — рейс поезда с указанием даты и времени отправления и прибытия, точек отправления и прибытия (станции), стоимости билета, количества доступных мест и ссылки на поезд (TrainId).

12. Статус билета (Status) — состояние билета («Забронирован», «Оплачен», «Возврат»).

13. Билет (Ticket) — основной документ, удостоверяющий право пассажира на проезд. Содержит ссылку на пассажира (PassengerId), дату покупки, статус, ссылку на маршрут (TripId), ссылку на место (PlaceId) и комментарий.

14. Детский билет (ChildTicket) — дополнительная сущность для оформления билетов на детей (без отдельного места или с льготным тарифом). Содержит фамилию, имя, отчество, дату рождения ребёнка, ссылку на основной билет (TicketId), ссылку на место (PlaceId) и свидетельство о рождении (SVoRojd).

Типы связей:

1. Должность (Post) – Сотрудник (User) — один-ко-многим. Множество сотрудников могут иметь одну должность.

2. Пассажир (User) – Паспорт (Passports) — один-ко-многим. У одного пассажира может быть несколько паспортов (теоретически, при смене документа). В текущей логике — чаще один к одному.

3. Город (City) – Станция (RailwayStation) — один-ко-многим. В одном городе может быть несколько железнодорожных станций.

4. Поезд (Train) – Вагон (Van) — многие-ко-многим. Один поезд включает множество вагонов, один вагон может входить в состав разных поездов. Связь реализована через таблицу TrainVan.

5. Вагон (Van) – Место (Place) — многие-ко-многим. В одном вагоне множество мест, одно место может быть в разных вагонах (абстрактно). Связь реализована через таблицу VanPlace.

6. Тип вагона (TypeCarriage) – Вагон (Van) — один-ко-многим. Множество вагонов могут относиться к одному типу.

7. Станция отправления (RailwayStation) – Маршрут (Trip) — один-ко-многим (как начальный пункт). На одной станции может начинаться множество маршрутов.

8. Станция прибытия (RailwayStation) – Маршрут (Trip) — один-ко-многим (как конечный пункт). На одной станции может заканчиваться множество маршрутов.

9. Поезд (Train) – Маршрут (Trip) — один-ко-многим. Один поезд может выполнять множество рейсов (маршрутов) в разное время.

10. Пассажир (User) – Билет (Ticket) — один-ко-многим. Один пассажир может приобрести множество билетов.

11. Статус (Status) – Билет (Ticket) — один-ко-многим. Множество билетов могут иметь один статус.

12. Маршрут (Trip) – Билет (Ticket) — один-ко-многим. На один маршрут может быть оформлено множество билетов.

13. Место (Place) – Билет (Ticket) — один-ко-многим. Одно место может быть занято в разных рейсах (в разное время), что отражается в билетах.

14. Билет (Ticket) – Детский билет (ChildTicket) — один-ко-многим. К одному взрослому билету может быть привязано несколько детских билетов (без отдельных мест или с местами).

2.1.2. Разработка физической модели данных

Цель физического проектирования – преобразование логической модели с учетом синтаксиса, семантики и возможностей выбранной целевой СУБД.

В проекте используется MS SQL Server - высокопроизводительная, надежная и масштабируемая система управления реляционными базами данных. Выбор обоснован следующими причинами:

1. Отказоустойчивость и масштабирование;
2. Клиент-серверная архитектура;
3. Отличная поддержка многопользовательского режима;
4. Высокая производительность;

Описание структур таблиц приведено в таблицах 1–15.

Таблица 1. Структура таблицы «Должность»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код должности	PostId	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор должности
Название	PostName	NVARCHAR R(25)	NOT NULL	Наименование должности

Таблица 2. Структура таблицы «Пользователи»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код пользователя	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор пользователя
Имя	FirstName	NVARCHAR R(50)	NOT NULL	Имя пользователя
Фамилия	LastName	NVARCHAR R(50)	NOT NULL	Фамилия пользователя

Отчество	MiddleName	NVARCHAR R(50)	NULL	Отчество пользователя
Должность	Position	NVARCHAR R(100)	NOT NULL	Должность
Логин	Login	NVARCHAR R(20)	NULL	Учётная запись для входа в систему
Пароль	Password	NVARCHAR R(20)	NULL	Пароль для авторизации
Фото	Photo	NVARCHAR R(MAX)	NULL	Путь к фотографии пользователя
Код должности	PostId	INT	Foreign Key (Post), NOT NULL	Определяет роль в системе

Таблица 3. Структура таблицы «Паспорт»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код паспорта	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор паспорта
Серия паспорта	PassportSeries	NVARCHAR R(4)	NULL	Серия паспорта
Номер паспорта	PassportNumber	NVARCHAR R(6)	NULL	Номер паспорта
Кем выдан	IssuedBy	NVARCHAR R(100)	NULL	Орган, выдавший паспорт
Дата выдачи	IssueDate	DATE	NULL	Дата выдачи паспорта
Код пассажира	PassengerId	INT	Foreign Key (User), NULL	Ссылка на владельца паспорта

Таблица 4. Структура таблицы «Город»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код города	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор города
Название города	Name	NVARCHAR R(25)	NOT NULL	Наименование населённого пункта

Таблица 5. Структура таблицы «Станция»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код станции	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор станции

Название станции	Name	NVARCHAR R(100)	NOT NULL, UNIQUE	Наименование станции
Код города	CityId	INT	Foreign Key (City), NULL	Ссылка на город, в котором находится станция

Таблица 6. Структура таблицы «Тип вагона»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код типа вагона	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор типа вагона
Название типа вагона	Name	NVARCHAR R(100)	NOT NULL	Тип вагона

Таблица 7. Структура таблицы «Поезд»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код поезда	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор поезда
Название поезда	Name	NVARCHAR R(100)	NULL	Наименование поезда

Таблица 8. Структура таблицы «Вагон»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код вагона	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор вагона
Название вагона	Name	NVARCHAR R(100)	NULL	Номер или наименование вагона
Код типа вагона	TypeCarId	INT	Foreign Key (TypeCarriage), NULL	Ссылка на тип вагона

Таблица 9. Структура таблицы «Место»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код места	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор места
Название места	Name	NVARCHAR R(100)	NULL	Номер/обозначение места

Таблица 10. Структура таблицы «Состав поезда»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код состава поезда	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор состава поезда
Код поезда	TrainId	INT	Foreign Key (Train), NULL	Ссылка на поезд
Код вагона	VanId	INT	Foreign Key (Van), NULL	Ссылка на вагон, входящий в состав поезда

Таблица 11. Структура таблицы «Схема вагона»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код схемы вагона	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор схемы вагона
Код вагона	VanId	INT	Foreign Key (Van), NULL	Ссылка на вагон
Код места	PlaceId	INT	Foreign Key (Place), NULL	Ссылка на место, находящееся в данном вагоне

Таблица 12. Структура таблицы «Статус билета»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код статуса	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор статуса
Название статуса	Name	NVARCHAR(25)	NOT NULL	Статус билета

Таблица 13. Структура таблицы «Маршрут / Поездка»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код маршрута	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор маршрута
Дата и время отправления места	StartDate	DATETIME	NULL	Дата и время отхода поезда от станции отправления
Дата и время прибытия	EndDate	DATETIME	NULL	Дата и время прибытия на станцию назначения
Станция отправления	StartPoint	INT	Foreign Key (RailwayStation), NULL	Ссылка на станцию отправления

Станция прибытия	EndPoint	INT	Foreign Key (RailwayStation), NULL	Ссылка на станцию прибытия
Стоимость билета	Price	MONEY	NULL	Цена билета на данный рейс
Количество доступных мест	CountT	INT	NULL	Число свободных мест в поездке
Код поезда	TrainId	INT	Foreign Key (Train), NULL	Ссылка на поезд, выполняющий рейс

Таблица 14. Структура таблицы «Билет»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код билета	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор билета
Код пассажира	Name	NVARCHAR R(100)	NULL	Ссылка на пассажира
Дата покупки	DateBuy	DATETIME	NULL	Дата и время оформления билета
Код статуса	Status	INT	Foreign Key (Status), NULL	Текущий статус билета
Код маршрута	TripId	INT	Foreign Key (Trip), NULL	Ссылка на рейс
Код места	PlaceId	INT	Foreign Key (Place), NULL	Ссылка на конкретное место в вагоне
Комментарий	Comment	NVARCHAR R(100)	NULL	Дополнительная информация

Таблица 15. Структура таблицы «Детский билет»

Название поля	Идентификатор	Тип данных	Условия и ограничения	Описание
Код детского билета	Id	INT	Primary Key, IDENTITY(1,1), NOT NULL	Идентификатор детского билета
Фамилия	Surname	NVARCHAR R(100)	NULL	Фамилия ребёнка
Имя	Name	NVARCHAR R(100)	NULL	Имя ребёнка
Отчество	Patronymic	NVARCHAR R(100)	NULL	Отчество ребёнка
Дата рождения	Birthday	DATE	NULL	Дата рождения ребёнка
Код основного билета	TicketId	INT	Foreign Key (Ticket), NULL	Ссылка на билет, к которому привязан ребёнок

Код места	PlaceId	INT	Foreign Key (Place), NULL	Место ребёнка
Свидетельство о рождении	SVoRojd	NVARCHAR R(100)	NULL	Скан или путь к документу

2.1.3. Листинг SQL операторов

После создания общей структуры базы данных можно приступить к созданию таблиц, которые представляют собой отношения, входящие в состав проекта базы данных.

Листинг сценария создания таблиц представлен в приложении 6.

Результат работы запросов представлен на рисунке 1.

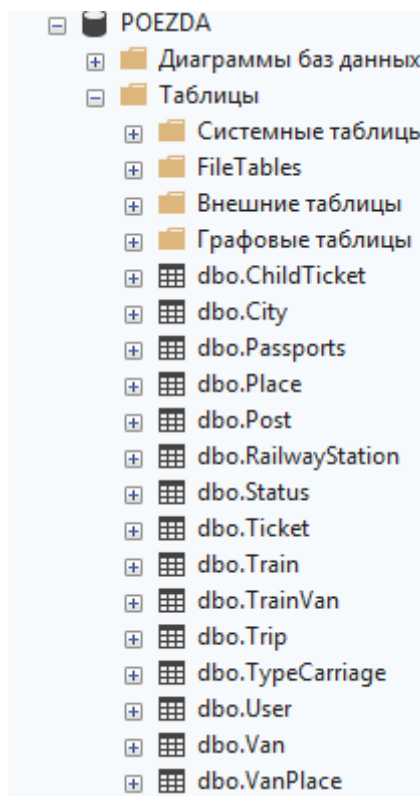


Рис. 1

2.2. Разработка клиентской части приложения

2.2.1. Разработка структуры приложения

Структура приложения представлена в приложении 4.

2.2.2. Разработка пользовательского интерфейса приложения

Интерфейс приложения представлен на рисунках 2-3.



Добро пожаловать!

Войдите в свой аккаунт

Логин

Пароль

 Войти

или

 Регистрация

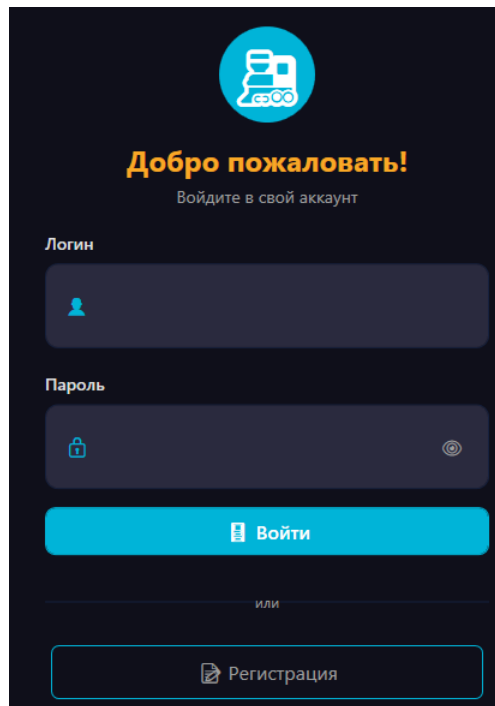


Рис. 2 Страница авторизации

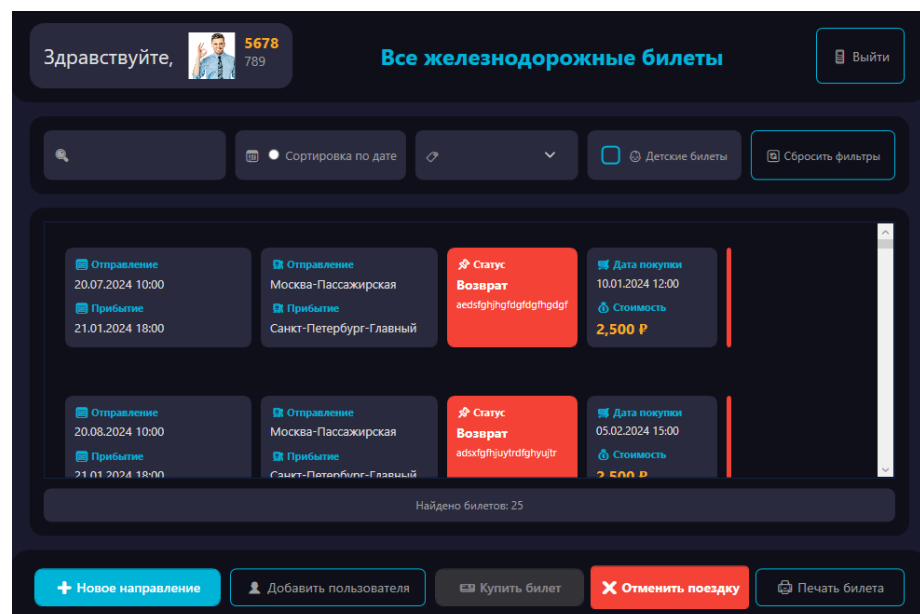


Рис. 3 Основная страница

2.2.3. Разработка диалога авторизации приложения

Страница авторизации содержит следующий функционал:

- Ввод логина/пароля
- Кнопка авторизации и перехода на основную страницу

Главная страница содержит следующий функционал:

- Выход из сессии и переход на страницу авторизации

- Переключение между страницами основных сущностей

Страница заказов содержит следующий функционал:

- Просмотр билетов
- Поиск/фильтрация/сортировка по маршрутам
- Открытие окна создания нового маршрута
- Открытие окна редактирования маршрута
- Удаление маршрута

2.2.4. Разработка программных модулей приложения

Окна создания/редактирования маршрутов содержат следующий функционал:

- Выбор вокзала отправления
- Выбор вокзала прибытия
- Выбор даты маршрута
- Выбор способа оплаты
- Сохранение/отмена создания/редактирования

Окна создания/редактирования маршрутов содержат следующий функционал:

- Выбор даты отправления
- Выбор даты прибытия
- Выбор вокзала отправления
- Выбор вокзала прибытия
- Выбор поезда
- Ввод цены
- Ввод количества мест
- Сохранение/отмена создания/редактирования

Окна создания/редактирования работника содержат следующий функционал:

- Ввод фамилии
- Ввод имени
- Ввод отчества
- Ввод логина
- Ввод пароля
- Выбор должности
- Выбор фото
- Сохранение/отмена создания/редактирования

Окна создания/редактирования клиента содержат следующий функционал:

- Ввод названия
- Выбор типа категории
- Ввод описания
- Сохранение/отмена создания/редактирования

Окна создания/редактирования клиента содержат следующий функционал:

- Ввод фамилии
- Ввод имени
- Ввод отчества
- Ввод должности
- Ввод телефона
- Ввод email
- Выбор даты найма
- Сохранение/отмена создания/редактирования

Страница билетов содержит следующий функционал:

- Просмотр списка билетов
- Открытие окна создания нового маршрута
- Открытие окна редактирования маршрута
- Удаление маршрута

2.2.5. Разработка отчетных форм приложения

Создание отчетной формы представлено в приложении 5.

2.2.6. Разработка тестов приложения

Тестирование программы — процесс исследования, испытания программного продукта, имеющий две различные цели:

- продемонстрировать разработчикам и заказчикам, что программа соответствует требованиям;
- выявить ситуации, в которых поведение программы является неправильным, нежелательным или не соответствующим спецификации.

Список тестов представлен в таблицах 16-20.

Таблица 16. Тестирование авторизации

Наименование	Описание
Test Case #1	ТС_UI_1
Приоритет теста	Высокий
Название тестирования	Проверка отлова ввода некорректных данных на странице авторизации
Набор входных данных	Логин: admin Пароль: qwerty
Шаги	1. Открыть страницу авторизации 2. Ввести тестовые данные 3. Нажать на кнопку «Войти»
Ожидаемый результат	Вход в систему не выполняется. Выводится сообщение с ошибкой «Такого пользователя нет в системе»
Фактический результат	Вход в систему не выполняется. Выводится сообщение с ошибкой «Такого пользователя нет в системе»

Таблица 17. Тестирование Создания заказа

Наименование	Описание
Test Case #2	ТС_UI_2
Приоритет теста	Средний
Название тестирования	Проверка отлова создания неполностью заполненного маршрута
Набор входных данных	Дата отправления: *пуст* Дата прибытия: *пуст* Вокзал отправления: Санкт-Петербург - Главный Вокзал прибытия: Москва - Пассажирская Поезд: Класный поезд Цена: 15151 Количество мест: 65465
Шаги	1. Выполнить вход в систему 2. Перейти на страницу маршрутов 3. Открыть окно создания нового маршрута 4. Заполнить поля тестовыми данными Нажать на кнопку «Добавить»
Ожидаемый результат	Сохранение не происходит. Выводится сообщение с ошибкой «Выберите даты отправления и прибытия»
Фактический результат	Сохранение не происходит. Выводится сообщение с ошибкой «Выберите даты отправления и прибытия»

Таблица 18. Тестирование удаления заказа

Наименование	Описание
Test Case #3	ТС_UI_3
Приоритет теста	Высокий
Название тестирования	Проверка отлова попытки удаления маршрута без выбора элемента из списка
Набор входных данных	Отсутствуют
Шаги	1. Выполнить вход в систему 2. Перейти на страницу маршрутов 3. Не выделяю запись в списке нажать на кнопку «Удалить»
Ожидаемый результат	Удаление не происходит. Кнопка недоступна
Фактический результат	Удаление не происходит. Кнопка недоступна

Таблица 19. Тестирование создания нового пользователя

Наименование	Описание
Test Case #4	ТС_UI_4
Приоритет теста	Средний
Название тестирования	Проверка работоспособности формы создания нового пользователя в системе
Набор входных данных	Имя: Анна Фамилия: Петрова Отчество: Владимировна Логин: Anna Пароль: 123321Ya! Должность: Менеджер
Шаги	1. Выполнить вход в систему 2. Перейти на страницу пользователей 3. Открыть окно создания нового пользователя 4. Заполнить поля тестовыми данными 5. Нажать на кнопку «Добавить»
Ожидаемый результат	Пользователь успешно создается. Окно создания нового пользователя закрывается и выводится список билетов.
Фактический результат	Пользователь успешно создается. Окно создания нового пользователя закрывается и выводится список билетов.

2.2.7. Результаты работы программы и программных сообщений

Вывод ошибки при отсутствии введенных учетных данных в базе представлен на рисунке 4.

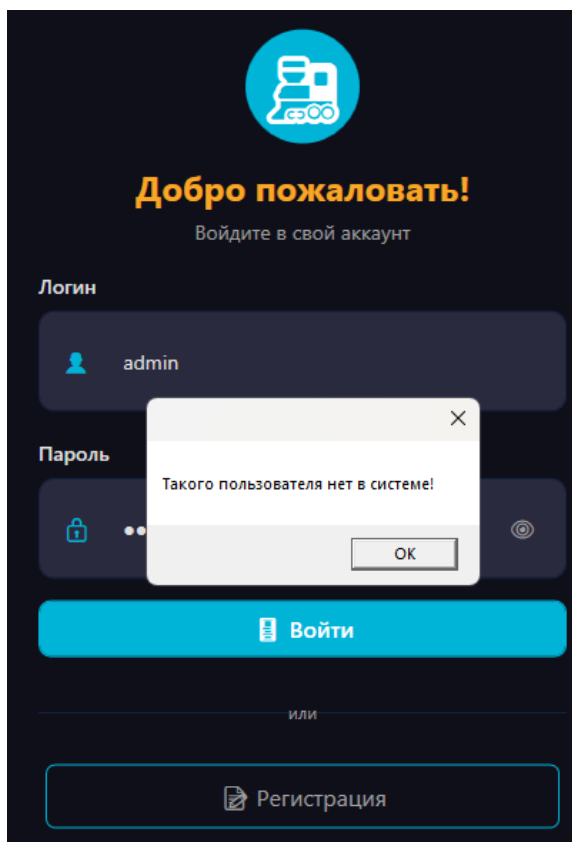


Рис. 4. Ошибка при отсутствии введенных учетных данных в базе.

Вывод сообщения об ошибке при оставлении маршрута неполным представлен на рисунке 5.

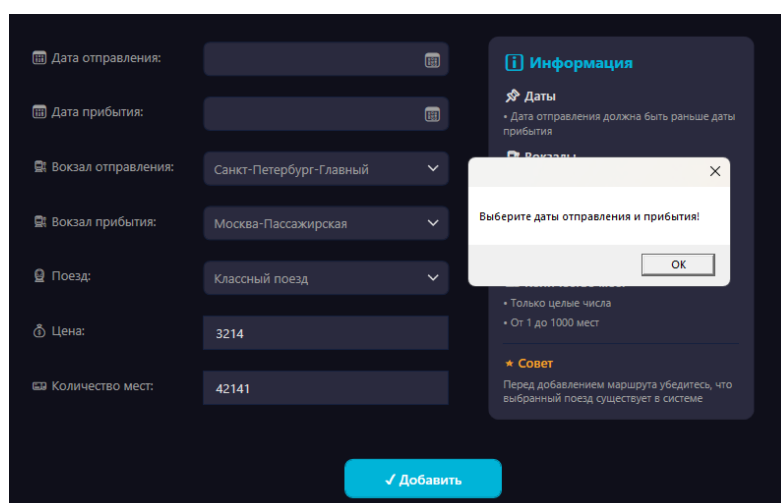


Рис. 5 Ошибка при оставлении маршрута неполным.

Вывод списка билетов представлен на рисунке 6.

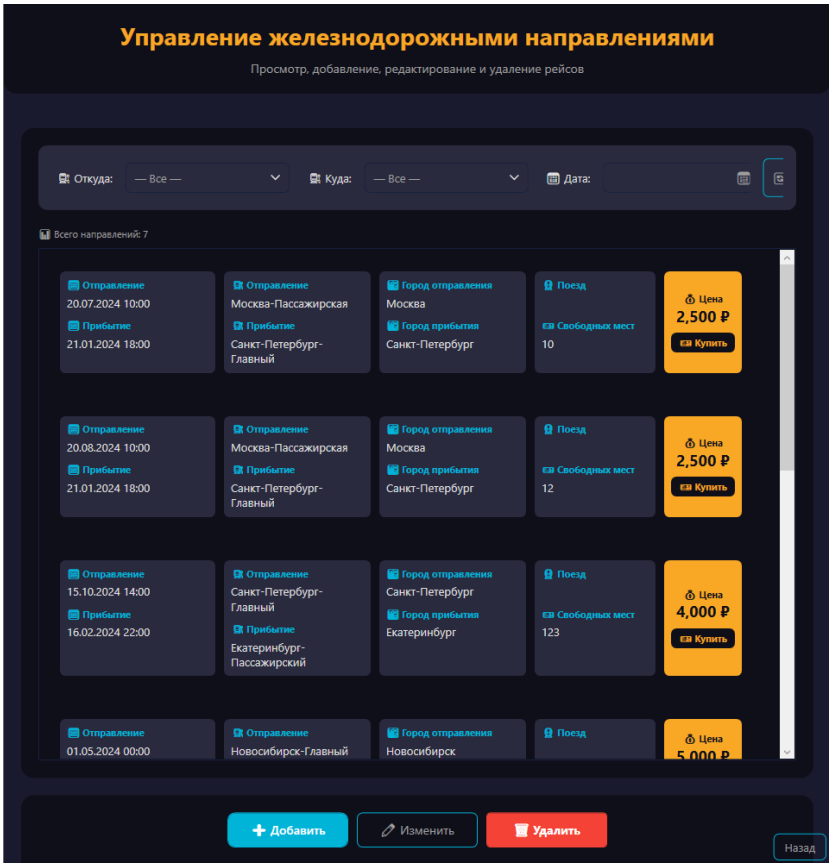


Рис. 6. Вывод списка билетов.

Вывод сообщения об успешном создании нового пользователя представлен на рисунке 7.

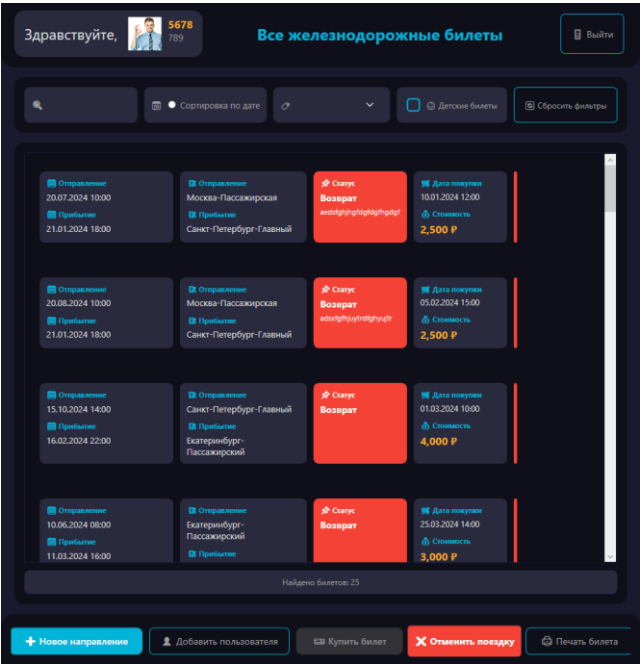


Рис. 7. Вывод билетов после создания пользователя.

Печать сгенерированного билета представлен на рисунке 8.

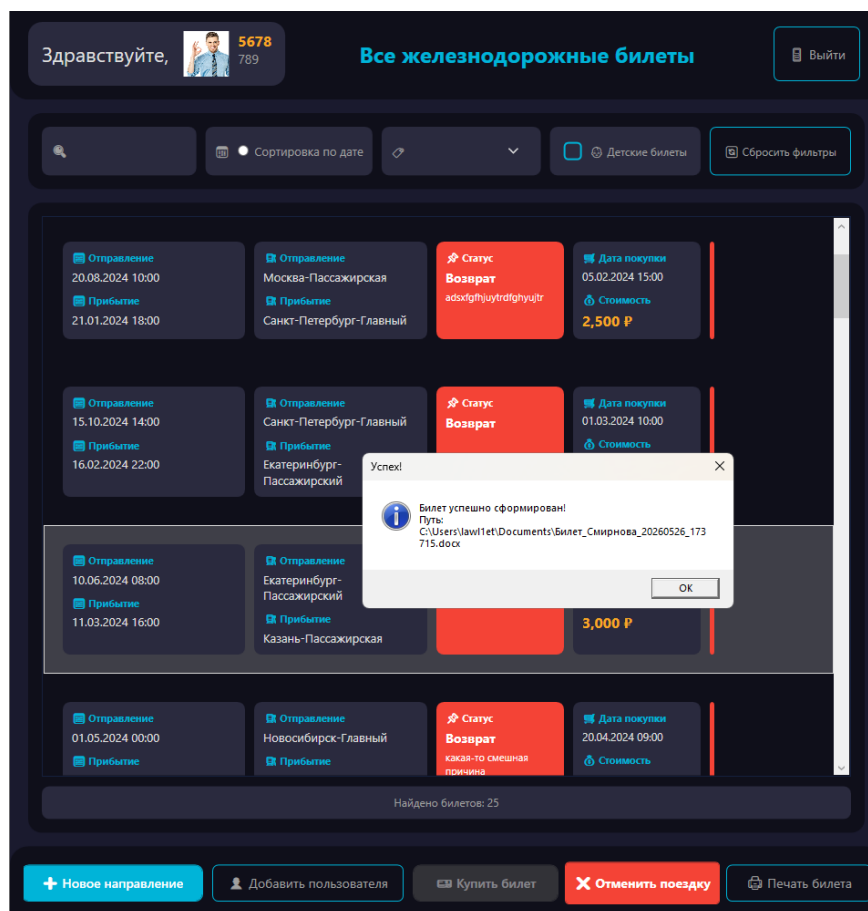


Рис. 8. Печать билета.

2.3. Разработка руководства пользователя приложения

Для входа в систему необходимо на странице авторизации ввести логин и пароль от своей учетной записи и нажать на кнопку «Войти»

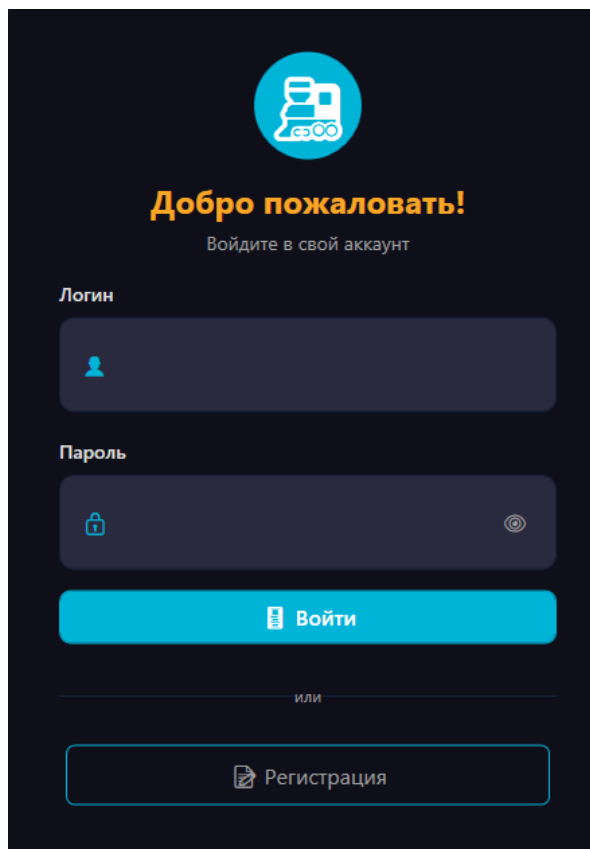


Рис. 9. Страница авторизации.

2.3.1. Администратор

Администратор может создавать/редактировать/удалять записи в следующих справочниках:

- Билеты;
- Маршруты;
- Пользователи;

Для создания нового маршрута необходимо перейти на страницу маршрутов и нажать на кнопку соответствующую кнопки добавления. Откроется окно добавления нового маршрута. Необходимо заполнить все поля. После заполнения формы нажать на кнопку «Добавить»

📅 Дата отправления:

📅

📅 Дата прибытия:

📅

🚉 Вокзал отправления:

▼

🚉 Вокзал прибытия:

▼

🚆 Поезд:

▼

💰 Цена:

🛋️ Количество мест:

ℹ️ **Информация**

📅 **Даты**

• Дата отправления должна быть раньше даты прибытия

🚉 **Вокзалы**

• Вокзал отправления и прибытия не должны совпадать

💰 **Цена**

• Только цифры
• От 1 до 100000 рублей

🛋️ **Количество мест**

• Только целые числа
• От 1 до 1000 мест

★ **Совет**

Перед добавлением маршрута убедитесь, что выбранный поезд существует в системе

✓ **Добавить**

Рис. 10. Создание новой записи на примере товаров.

В случае ошибки будет выведена ошибка в диалоговом окне.

The screenshot shows a form for creating a new record. The form has several fields: "Дата отправления:" (Date of departure), "Дата прибытия:" (Date of arrival), "Вокзал отправления:" (Departure station), "Вокзал прибытия:" (Arrival station), "Поезд:" (Train), "Цена:" (Price), and "Количество мест:" (Number of seats). To the right of the form is a panel titled "Информация" (Information) with sections: "Даты" (Dates), "Вокзалы" (Stations), "Цена" (Price), and "Количество мест" (Number of seats). Each section contains a list of rules. A modal dialog box is displayed in the foreground with the text "Выберите даты отправления и прибытия!" (Select the dates of departure and arrival!) and an "OK" button.

Рис. 11. Пример ошибки при неправильном заполнении формы создания.

После успешного создания новой записи будет выведено соответствующее сообщение в диалоговом окне.

The screenshot shows the same form as in Figure 11, but now it is filled with data: "Дата отправления:" is "28.04.2026", "Дата прибытия:" is "19.06.2026", "Вокзал отправления:" is "Санкт-Петербург-Главный", "Вокзал прибытия:" is "Казань-Пассажирская", "Поезд:" is "Классный поезд", "Цена:" is "421", and "Количество мест:" is "414". The modal dialog box now displays the text "Успешно добавлено!" (Successfully added!) and an "OK" button.

Рис. 12. Пример сообщения об успешном создании новой записи.

Для редактирования записи необходимо перейти на страницу справочника, выбрать маршрут из списка и нажать соответствующую кнопку «Изменить». Откроется окно, такого же наполнения как и при создании, но с

заполненными полями, данным выбранной записи. Необходимо отредактировать данные и нажать на кнопку «Сохранить»

Для удаления записи необходимо перейти на страницу справочника, выбрать запись из списка и нажать соответствующую кнопку «Удалить». Появится диалоговое окно подтверждения действия. Необходимо нажать на кнопку «Да» после чего запись будет удалена.

2.3.2. Сотрудник

Сотрудник имеет доступ в следующим справочниках:

- Билеты;
- Маршруты;
- Пользователи;

Инструкция к работе «Сотрудника» аналогична инструкции для «Администратора», но с некоторыми правками, такими как: недоступно удаление товаров.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была достигнута поставленная цель — создание приложения для авторизации продаж железнодорожных билетов. Была исследована специфика предметной области, спроектирована база данных, разработано клиентское приложение для работы с базой данных, создано руководство пользователя.

Я приобрел, практические навыки использования среды программирования Visual Studio C# 2026 и применил на практике:

- создание клиент-серверного приложения БД;
- создание отчётов;
- вывод отчета на печать.

Закрепил практические навыки разработки БД в СУБД MS SQL Server:

- создание базы данных;
- создание структур таблиц баз данных;
- ввод тестовых данных в таблицы базы данных;

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Работа с T-SQL в Microsoft SQL Server. [Онлайн]. Доступно: <https://docs.microsoft.com/ru-ru/sql/t-sql/language-reference?view=sql-server-ver15>. Дата обращения: 25 мая 2026 г.
2. Руководство по использованию ADO.NET для работы с базой данных в C#. [Онлайн]. Доступно: <https://www.c-sharpcorner.com/UploadFile/84c85b/adonet-in-C-Sharp/>. Дата обращения: 25 мая 2023 г.
3. Работа с ADO.NET Entity Framework в C#. [Онлайн]. Доступно: <https://www.entityframeworktutorial.net/efcore/entity-framework-core.aspx>. Дата обращения: 25 мая 2026 г.
4. Введение в T-SQL для разработчиков C#. [Онлайн]. Доступно: <https://www.sqlservertutorial.net/ru/sql-tutorial/>. Дата обращения: 28 мая 2026 г.
5. Документация по Windows Presentation Foundation. [Онлайн]. Доступно: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/>. Дата обращения: 25 мая 2026 г.
6. Сайт вопросов и ответов о программировании. [Онлайн]. Доступно: <https://stackoverflow.com/questions/tagged/wpf?tab=Votes>. Дата обращения: 27 мая 2026 г.

ПРИЛОЖЕНИЕ 1

13	1	2024-01-10 12:0...	3	4	NULL	aedsfghjhgfdgf...
14	2	2024-02-05 15:0...	3	6	NULL	adsxfgfhjuytrdf...
15	3	2024-03-01 10:0...	3	7	NULL	NULL
16	4	2024-03-25 14:0...	3	8	NULL	NULL

Рис. 13. Входные данные.

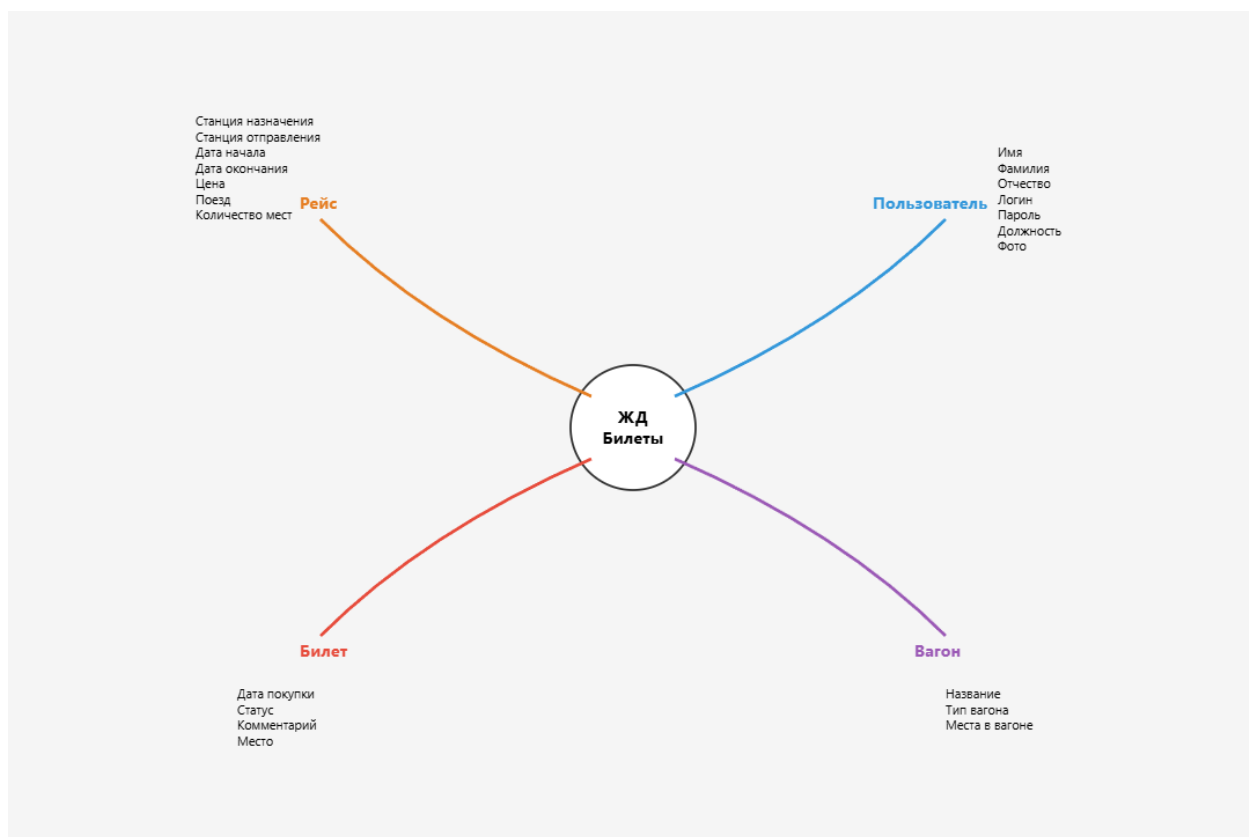


Рис. 14. Интеллектуальная карта (концептуальная модель).

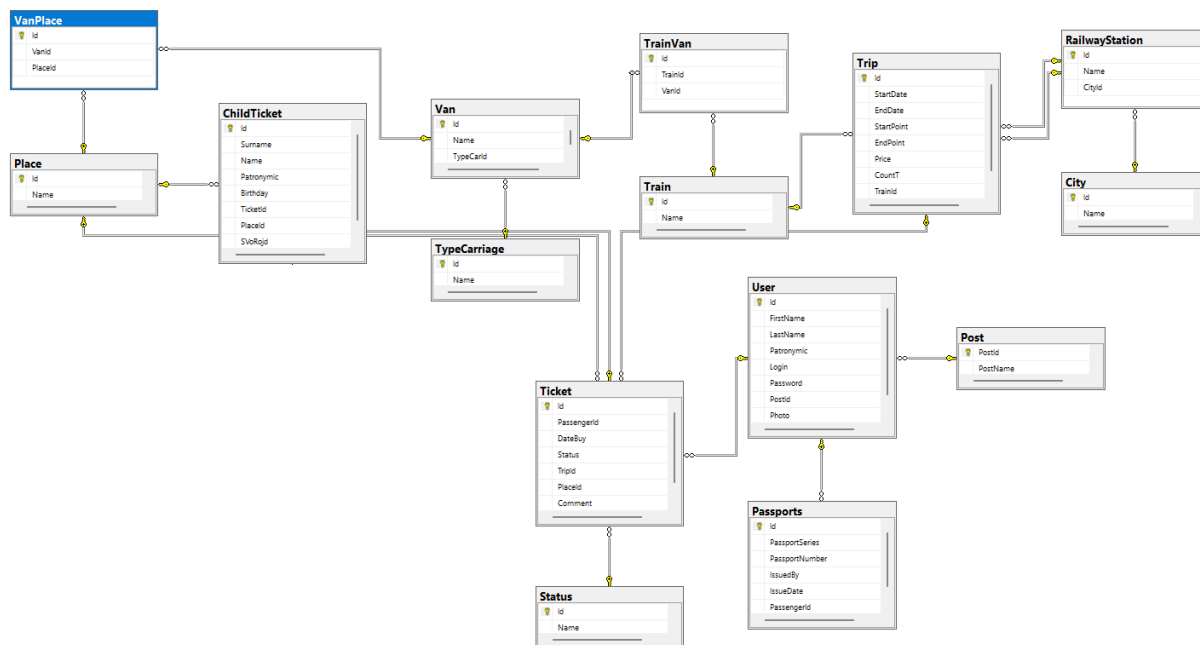


Рис. 15. Логическая модель данных (ER-диаграмма).

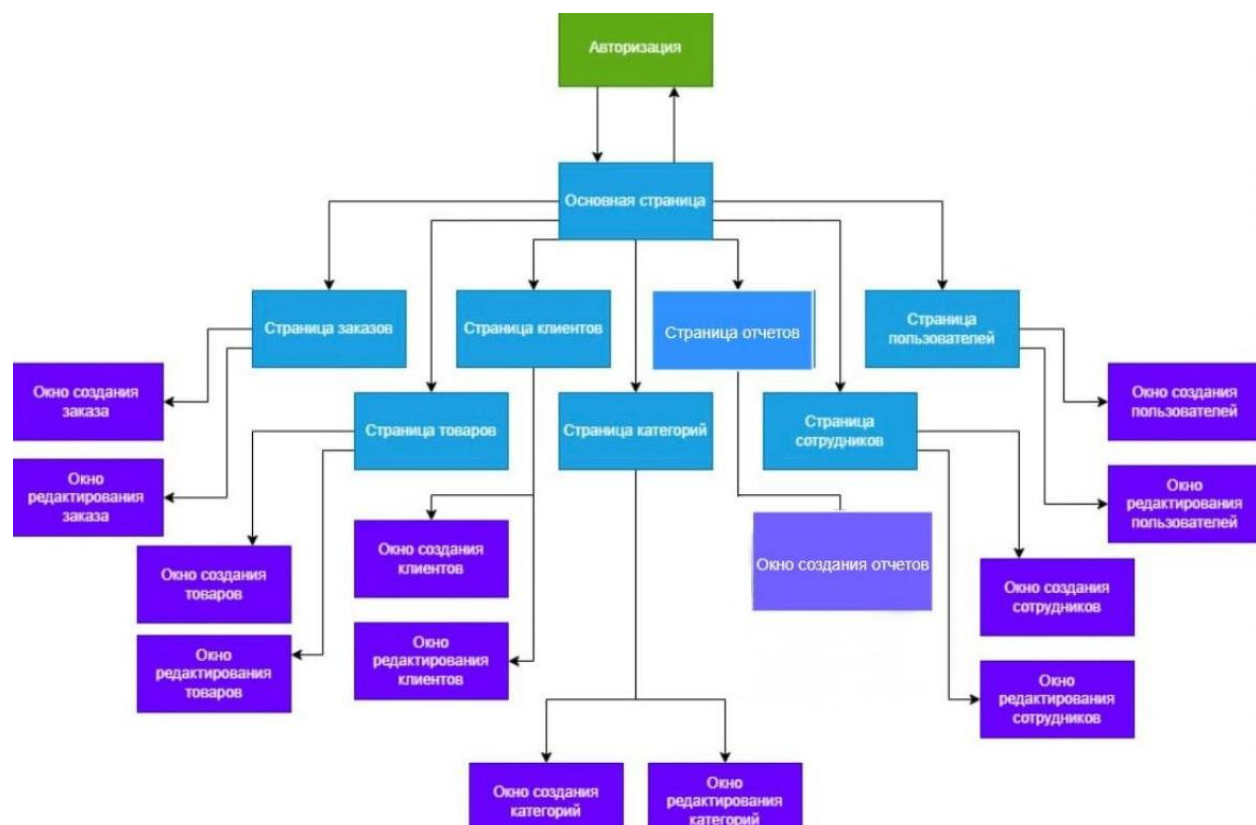


Рис. 16. Структурная схема приложения

ОАО «РОССИЙСКИЕ ЖЕЛЕЗНЫЕ ДОРОГИ» ЭЛЕКТРОННЫЙ ПРОЕЗДНОЙ ДОКУМЕНТ	
Направление:	Москва-Пассажирская → Санкт-Петербург-Главный
Дата отправления:	вторник, 20 августа 2024
Время отправления:	10:00
Время прибытия:	18:00
Пассажир:	Иванов Иван Иванович
Место:	Вагон: —, Место: 0
Стоимость:	4 750,00 руб.
00000043	
Внимание! П посадка на поезд осуществляется при предъявлении документа, удостоверяющего личность, и данного билета (на бумажном носителе или в электронном виде).	

Рис. 17. Печатная форма приложения

```

USE master;
GO

IF EXISTS (SELECT name FROM sys.databases WHERE name = 'BlinShopDB')
BEGIN
    ALTER DATABASE BlinShopDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE BlinShopDB;
END
GO

CREATE DATABASE BlinShopDB;
GO

USE BlinShopDB;
GO

CREATE TABLE Role
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(50) NOT NULL UNIQUE,
    Description NVARCHAR(500) NULL
);
GO

CREATE TABLE Employee
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    MiddleName NVARCHAR(50) NULL,
    Position NVARCHAR(100) NOT NULL,
    Phone NVARCHAR(20) NULL,
    Email NVARCHAR(100) NULL,
    HireDate DATE NOT NULL DEFAULT GETDATE(),
    IsActive BIT NOT NULL DEFAULT 1
);
GO

CREATE TABLE [User]
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    EmployeeId INT NOT NULL,
    RoleId INT NOT NULL,
    Login NVARCHAR(50) NOT NULL UNIQUE,
    Password NVARCHAR(255) NOT NULL,
    CONSTRAINT FK_User_Employee FOREIGN KEY (EmployeeId) REFERENCES
Employee(Id),
    CONSTRAINT FK_User_Role FOREIGN KEY (RoleId) REFERENCES Role(Id),
    CONSTRAINT UQ_User_EmployeeId UNIQUE (EmployeeId)
);
GO

CREATE TABLE Customer
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    MiddleName NVARCHAR(50) NULL,
    Phone NVARCHAR(20) NULL,
    Email NVARCHAR(100) NULL,
    Login NVARCHAR(50) NULL,
    Password NVARCHAR(255) NULL
);
GO

CREATE TABLE Category
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500) NULL

```

Рис. 18. Листинг скрипта создания базы данных

```

);
GO
CREATE TABLE Product
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CategoryId INT NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500) NULL,
    Composition NVARCHAR(1000) NULL,
    Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
    ImageUrl NVARCHAR(500) NULL,
    CONSTRAINT FK_Product_Category FOREIGN KEY (CategoryId) REFERENCES
Category(Id)
);
GO

CREATE TABLE [Order]
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CustomerId INT NULL,
    EmployeeId INT NOT NULL,
    OrderDate DATETIME2 NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),
    PaymentMethod NVARCHAR(50) NOT NULL,
    Status NVARCHAR(50) NOT NULL DEFAULT N'Ожидает',
    CONSTRAINT FK_Order_Customer FOREIGN KEY (CustomerId) REFERENCES
Customer(Id),
    CONSTRAINT FK_Order_Employee FOREIGN KEY (EmployeeId) REFERENCES
Employee(Id)
);
GO

CREATE TABLE OrderItem
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    OrderId INT NOT NULL,
    ProductId INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice >= 0),
    Subtotal DECIMAL(10,2) NOT NULL CHECK (Subtotal >= 0),
    CONSTRAINT FK_OrderItem_Order FOREIGN KEY (OrderId) REFERENCES
[Order](Id) ON DELETE CASCADE,
    CONSTRAINT FK_OrderItem_Product FOREIGN KEY (ProductId) REFERENCES
Product(Id)
);
GO

CREATE TABLE Receipt
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    OrderId INT NOT NULL,
    ReceiptNumber NVARCHAR(50) NOT NULL UNIQUE,
    ReceiptDate DATETIME2 NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),
    CONSTRAINT FK_Receipt_Order FOREIGN KEY (OrderId) REFERENCES [Order](Id),
    CONSTRAINT UQ_Receipt_OrderId UNIQUE (OrderId)
);
GO

CREATE INDEX IX_Role_Name ON Role(Name);
CREATE INDEX IX_Employee_IsActive ON Employee(IsActive);
CREATE INDEX IX_Employee_HireDate ON Employee(HireDate);
CREATE INDEX IX_User_EmployeeId ON [User](EmployeeId);
CREATE INDEX IX_User_RoleId ON [User](RoleId);
CREATE INDEX IX_User_Login ON [User](Login);

CREATE INDEX IX_Customer_Phone ON Customer(PHONE);
CREATE UNIQUE INDEX IX_Customer_Login ON Customer(Login) WHERE Login IS NOT
NULL;

```

Рис. 18. Листинг скрипта создания базы данных


```

        CONSTRAINT FK_Product_Category FOREIGN KEY (CategoryId) REFERENCES
        Category(Id),
        CONSTRAINT FK_Product_Supplier FOREIGN KEY (SupplierId) REFERENCES
        Supplier(Id)
    );
GO

CREATE TABLE [Order]
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CustomerId INT NULL,
    EmployeeId INT NOT NULL,
    OrderDate DATETIME2 NOT NULL DEFAULT GETDATE(),
    DeliveryDate DATETIME2 NULL,
    DeliveryAddress NVARCHAR(500) NULL,
    TotalAmount DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),
    PaymentMethod NVARCHAR(50) NOT NULL,
    Status NVARCHAR(50) NOT NULL DEFAULT 'Ожидает',
    CONSTRAINT FK_Order_Customer FOREIGN KEY (CustomerId) REFERENCES
    Customer(Id),
    CONSTRAINT FK_Order_Employee FOREIGN KEY (EmployeeId) REFERENCES
    Employee(Id)
);
GO

CREATE TABLE OrderItem
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    OrderId INT NOT NULL,
    ProductId INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice >= 0),
    Subtotal DECIMAL(10,2) NOT NULL CHECK (Subtotal >= 0),
    CONSTRAINT FK_OrderItem_Order FOREIGN KEY (OrderId) REFERENCES
    [Order](Id) ON DELETE CASCADE,
    CONSTRAINT FK_OrderItem_Product FOREIGN KEY (ProductId) REFERENCES
    Product(Id)
);
GO

CREATE TABLE Cheque
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    OrderId INT NOT NULL,
    ChequeNumber NVARCHAR(50) NOT NULL UNIQUE,
    ChequeDate DATETIME2 NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),
    CONSTRAINT FK_Cheque_Order FOREIGN KEY (OrderId) REFERENCES [Order](Id),
    CONSTRAINT UQ_Cheque_OrderId UNIQUE (OrderId)
);
GO

CREATE INDEX IX_Role_Name ON Role(Name);

```

Рис. 19. Листинг скрипта создания базы данных

ПРОДОЛЖЕНИЕ ПРИЛОЖЕНИЯ 6

```
CREATE INDEX IX_Order_OrderDate ON [Order](OrderDate);
CREATE INDEX IX_Order_Status ON [Order](Status);

CREATE INDEX IX_OrderItem_OrderId ON OrderItem(OrderId);
CREATE INDEX IX_OrderItem_ProductId ON OrderItem(ProductId);

CREATE INDEX IX_Receipt_OrderId ON Receipt(OrderId);
CREATE INDEX IX_Receipt_ReceiptNumber ON Receipt(ReceiptNumber);
CREATE INDEX IX_Receipt_ReceiptDate ON Receipt(ReceiptDate);
GO

PRINT 'Database BlinShopDB created successfully!';
PRINT 'All tables created with proper relationships and indexes.';
GO
```

Рис. 20. Листинг скрипта создания базы данных

```

public class DatabaseConnection
{
    private static string? _connectionString;

    public static string ConnectionString
    {
        get
        {
            if (string.IsNullOrEmpty(_connectionString))
            {
                try
                {
                    var config = ConfigurationManager.OpenExeConfiguration(ConfigurationUserLevel.None);
                    _connectionString = config.ConnectionStrings?.ConnectionStrings["DefaultConnection"]?.ConnectionString;
                }
                catch
                {
                }

                if (string.IsNullOrEmpty(_connectionString))
                {
                    _connectionString = "Data Source=.\SQLEXPRESS;Initial Catalog=BlinShopDB;Integrated Security=True;TrustServerCertificate=True";
                }
            }
            return _connectionString;
        }
    }

    public static SqlConnection GetConnection()
    {
        return new SqlConnection(ConnectionString);
    }
}

```

Рис. 21. Листинг DatabaseConnection.cs

```
public class BoolToVisibilityConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter,
        CultureInfo culture)
    {
        bool invert = string.Equals(parameter?.ToString(), "Invert",
            StringComparison.OrdinalIgnoreCase);

        if (value is bool boolValue)
        {
            if (invert)
            {
                boolValue = !boolValue;
            }

            return boolValue ? Visibility.Visible : Visibility.Collapsed;
        }
        return Visibility.Collapsed;
    }

    public object ConvertBack(object value, Type targetType, object
        parameter, CultureInfo culture)
    {
        if (value is Visibility visibility)
        {
            return visibility == Visibility.Visible;
        }
        return false;
    }
}
```

Рис. 22. Листинг BoolToVisibilityConverter.cs

```

public partial class CategoriesPage : Page
{
    public CategoriesPage()
    {
        InitializeComponent();
        LoadCategories();
    }

    private void LoadCategories()
    {
        try
        {
            using var connection = DatabaseConnection.GetConnection();
            connection.Open();
            string query = "SELECT Id, Name, Description FROM Category ORDER
BY Name";
            using var command = new SqlCommand(query, connection);
            using var adapter = new SqlDataAdapter(command);
            var dataTable = new DataTable();
            adapter.Fill(dataTable);
            var categories = new List<Category>();
            foreach (DataRow row in dataTable.Rows)
            {
                categories.Add(new Category
                {
                    Id = (int)row["Id"],
                    Name = row["Name"].ToString() ?? "",
                    Description = row["Description"] == DBNull.Value ? null :
row["Description"].ToString()
                });
            }
            CategoriesDataGrid.ItemsSource = categories;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка загрузки категорий: {ex.Message}",
"Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    private void AddCategoryButton_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            var dialog = new CategoryDialog();
            if (dialog.ShowDialog() == true)
            {
                LoadCategories();
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}

```

Рис. 23. Листинг CategoriesPage.cs

```

private void EditCategoryButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        if (CategoriesDataGrid.SelectedItem is Category category)
        {
            var dialog = new CategoryDialog(category);
            if (dialog.ShowDialog() == true)
            {
                LoadCategories();
            }
        }
        else
        {
            MessageBox.Show("Выберите категорию для редактирования",
                "Информация", MessageBoxButton.OK, MessageBoxImage.Information);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

private void DeleteCategoryButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        if (CategoriesDataGrid.SelectedItem is Category category)
        {
            var result = MessageBox.Show(
                $"Вы уверены, что хотите удалить категорию\n{category.Name}\n?",
                "Подтверждение удаления",
                MessageBoxButton.YesNo,
                MessageBoxImage.Question);
            if (result == MessageBoxResult.Yes)
            {
                using var connection = DatabaseConnection.GetConnection();
                connection.Open();
                // Проверяем, используется ли категория в товарах
                string checkQuery = "SELECT COUNT(*) FROM Product WHERE
CategoryId = @CategoryId";
                using var checkCommand = new SqlCommand(checkQuery,
                    connection);
                checkCommand.Parameters.AddWithValue("@CategoryId",
                    category.Id);
                int productCount = (int)checkCommand.ExecuteScalar();
                if (productCount > 0)
                {
                    MessageBox.Show(
                        "Невозможно удалить категорию, так как она
используется в товарах",
                        "Ошибка",
                        MessageBoxButton.OK,
                        MessageBoxImage.Warning);
                }
            }
            return;
        }
    }
}

```

Рис. 24. Листинг CategoriesPage.cs

```

        string deleteQuery = "DELETE FROM Category WHERE Id =
@Id";
        using var deleteCommand = new SqlCommand(deleteQuery,
connection);
        deleteCommand.Parameters.AddWithValue("@Id",
category.Id);
        deleteCommand.ExecuteNonQuery();

        MessageBox.Show("Категория успешно удалена", "Успех",
MessageBoxButton.OK, MessageBoxImage.Information);
        LoadCategories();
    }
    else
    {
        MessageBox.Show("Выберите категорию для удаления",
"Информация", MessageBoxButton.OK, MessageBoxImage.Information);
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка удаления: {ex.Message}", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
}

private void RefreshButton_Click(object sender, RoutedEventArgs e)
{
    LoadCategories();
}

private void CategoriesDataGrid_MouseDoubleClick(object sender,
System.Windows.Input.MouseButtonEventArgs e)
{
    EditCategoryButton_Click(sender, e);
}
}
}

```

Рис. 25. Листинг CategoriesPage.cs

```

private void LoadEmployees()
{
    try
    {
        using var connection = DatabaseConnection.GetConnection();
        connection.Open();
        string query = "SELECT Id, FirstName, LastName, MiddleName, Position,
Phone, Email, HireDate, IsActive FROM Employee ORDER BY LastName, FirstName";
        using var command = new SqlCommand(query, connection);
        using var adapter = new SqlDataAdapter(command);
        var dataTable = new DataTable();
        adapter.Fill(dataTable);
        var employees = new List<Employee>();
        foreach (DataRow row in dataTable.Rows)
        {
            employees.Add(new Employee
            {
                Id = (int)row["Id"],
                FirstName = row["FirstName"].ToString() ?? "",
                LastName = row["LastName"].ToString() ?? "",
                MiddleName = row["MiddleName"] == DBNull.Value ? null :
row["MiddleName"].ToString(),
                Position = row["Position"].ToString() ?? "",
                Phone = row["Phone"] == DBNull.Value ? null :
row["Phone"].ToString(),
                Email = row["Email"] == DBNull.Value ? null :
row["Email"].ToString(),
                HireDate = (DateTime)row["HireDate"],
                IsActive = (bool)row["IsActive"]
            });
        }
        EmployeesDataGrid.ItemsSource = employees;
        MessageBox.Show($"Ошибка загрузки сотрудников: {ex.Message}",
"Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

private void AddEmployeeButton_Click(object sender, RoutedEventArgs e)
{
    try
    {
        var dialog = new EmployeeDialog
        {
            Owner = Window.GetWindow(this)
        };
        if (dialog.ShowDialog() == true)
        {
            LoadEmployees();
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}

private void RefreshButton_Click(object sender, RoutedEventArgs e)
{
    LoadEmployees();
}

```

Рис. 25. Листинг EmployeesPage.cs


```

e) private void EditCustomerButton_Click(object sender, RoutedEventArgs
{
    try
    {
        if (CustomersDataGrid.SelectedItem is Customer customer)
        {
            var dialog = new CustomerDialog(customer);
            if (dialog.ShowDialog() == true)
            {
                LoadCustomers();
            }
        }
        else
        {
            MessageBox.Show("Выберите клиента для редактирования",
"Информация", MessageBoxButton.OK, MessageBoxImage.Information);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

e) private void DeleteCustomerButton_Click(object sender, RoutedEventArgs
{
    try
    {
        if (CustomersDataGrid.SelectedItem is Customer customer)
        {
            var result = MessageBox.Show(
                $"Вы уверены, что хотите удалить клиента
                \"{customer.FullName}\"?",
                "Подтверждение удаления",
                MessageBoxButton.YesNo,
                MessageBoxImage.Question);

            if (result == MessageBoxResult.Yes)
            {
                using var connection =
DatabaseConnection.GetConnection();
                connection.Open();

                // Проверяем, используется ли клиент в заказах
                string checkQuery = "SELECT COUNT(*) FROM [Order]
WHERE CustomerId = @CustomerId";
                using var checkCommand = new SqlCommand(checkQuery,
connection);
                checkCommand.Parameters.AddWithValue("@CustomerId",
customer.Id);

```

Рис. 27. Листинг CustomersPage.cs

```

public partial class OrdersPage : Page
{
    private readonly List<Order> _allOrders = new();

    private sealed class StatusFilterItem
    {
        public string? Value { get; init; }
        public string Name { get; init; } = string.Empty;
    }

    public OrdersPage()
    {
        InitializeComponent();
        ApplyRoleView();
        LoadOrders();
    }

    private void ApplyRoleView()
    {
        bool canManageOrders = AuthService.CanManageOrders();
        bool isClient = AuthService.IsClient();

        NewOrderButton.Visibility = canManageOrders ? Visibility.Visible :
        Visibility.Collapsed;
        EditOrderButton.Visibility = canManageOrders ? Visibility.Visible :
        Visibility.Collapsed;
        DeleteOrderButton.Visibility = canManageOrders ? Visibility.Visible :
        Visibility.Collapsed;

        if (isClient)
        {
            PageTitleTextBlock.Text = "Мои заказы";
            PageSubtitleTextBlock.Text = "Здесь отображаются только ваши
заказы и их текущие статусы.";
            GridTitleTextBlock.Text = "История ваших заказов";
        }
    }

    private void LoadOrders()
    {
        try
        {
            using var connection = DatabaseConnection.GetConnection();
            connection.Open();

            string query = @"
                SELECT o.Id, o.CustomerId, o.EmployeeId, o.OrderDate,
o.TotalAmount, o.PaymentMethod, o.Status,
                c.Id, c.FirstName, c.LastName, c.MiddleName, c.Phone,
c.Email, c.Login, c.Password,
                e.Id, e.FirstName, e.LastName, e.MiddleName,
e.Position, e.Phone, e.Email, e.HireDate, e.IsActive
                FROM [Order] o
                LEFT JOIN Customer c ON o.CustomerId = c.Id
                INNER JOIN Employee e ON o.EmployeeId = e.Id";

```

Рис. 27. Листинг CustomersPage.cs

```

null)
    if (AuthService.IsClient() && AuthService.CurrentCustomer !=
    {
        query += " WHERE o.CustomerId = @CustomerId";
    }

    query += " ORDER BY o.OrderDate DESC";

    using var command = new SqlCommand(query, connection);
    if (AuthService.IsClient() && AuthService.CurrentCustomer !=
null)
    {
        command.Parameters.AddWithValue("@CustomerId",
AuthService.CurrentCustomer.Id);
    }

    using var adapter = new SqlDataAdapter(command);
    var dataTable = new DataTable();
    adapter.Fill(dataTable);

    var orders = new List<Order>();
    foreach (DataRow row in dataTable.Rows)
    {
        var order = new Order
        {
            Id = (int)row["Id"],
            CustomerId = row["CustomerId"] == DBNull.Value ? null
: (int?)row["CustomerId"],
            EmployeeId = (int)row["EmployeeId"],
            OrderDate = (DateTime)row["OrderDate"],
            TotalAmount = (decimal)row["TotalAmount"],
            PaymentMethod = row["PaymentMethod"].ToString() ??
string.Empty,
            Status = row["Status"].ToString() ?? string.Empty
        };

        if (order.CustomerId.HasValue)
        {
            order.Customer = new Customer
            {
                Id = (int)row[7],
                FirstName = row[8].ToString() ?? string.Empty,
                LastName = row[9].ToString() ?? string.Empty,
                MiddleName = row[10] == DBNull.Value ? null :
row[10].ToString(),
                Phone = row[11] == DBNull.Value ? null :
row[11].ToString(),
                Email = row[12] == DBNull.Value ? null :
row[12].ToString(),
                Login = row[13] == DBNull.Value ? null :
row[13].ToString(),
                Password = row[14] == DBNull.Value ? null :
row[14].ToString()
            };
        }
    }

```

Рис. 28. Листинг MainWindow.xaml.cs

```

        order.Employee = new Employee
        {
            Id = (int)row[15],
            FirstName = row[16].ToString() ?? string.Empty,
            LastName = row[17].ToString() ?? string.Empty,
            MiddleName = row[18] == DBNull.Value ? null :
row[18].ToString(),
            Position = row[19].ToString() ?? string.Empty,
            Phone = row[20] == DBNull.Value ? null :
row[20].ToString(),
            Email = row[21] == DBNull.Value ? null :
row[21].ToString(),
            HireDate = (DateTime)row[22],
            IsActive = (bool)row[23]
        };

        orders.Add(order);
    }

    LoadOrderItemSummaries(connection, orders);

    _allOrders.Clear();
    _allOrders.AddRange(orders);

    PopulateStatusFilter(orders);
    ApplyFilters();
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка загрузки заказов: {ex.Message}",
"Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
}

private void PopulateStatusFilter(IEnumerable<Order> orders)
{
    string? selectedStatus = (StatusFilterComboBox.SelectedItem as
StatusFilterItem)?.Value;

    var items = new List<StatusFilterItem>
    {
        new() { Value = null, Name = "Все статусы" }
    };

    items.AddRange(orders
        .Where(order => !string.IsNullOrEmpty(order.Status))
        .Select(order => order.Status)
        .Distinct(StringComparer.OrdinalIgnoreCase)
        .OrderBy(status => status)
        .Select(status => new StatusFilterItem
        {
            Value = status,
            Name = status
        }
    ));
}

```

Рис. 29. Листинг MainWindow.cs

```

        StatusFilterComboBox.ItemsSource = items;
        StatusFilterComboBox.SelectedItem = items.FirstOrDefault(item =>
            string.Equals(item.Value, selectedStatus,
                StringComparison.OrdinalIgnoreCase)) ?? items[0];
    }

    private void ApplyFilters()
    {
        IEnumerable<Order> filteredOrders = _allOrders;
        string searchText = SearchTextBox.Text.Trim();

        if (!string.IsNullOrEmpty(searchText))
        {
            filteredOrders = filteredOrders.Where(order =>
                order.Id.ToString().Contains(searchText,
                    StringComparison.OrdinalIgnoreCase) ||
                ContainsText(order.Employee?.FullName, searchText) ||
                ContainsText(order.Status, searchText));
        }

        string? selectedStatus = (StatusFilterComboBox.SelectedItem as
            StatusFilterItem)?.Value;
        if (!string.IsNullOrEmpty(selectedStatus))
        {
            filteredOrders = filteredOrders.Where(order =>
                string.Equals(order.Status, selectedStatus,
                    StringComparison.OrdinalIgnoreCase));
        }

        if (StartDateFilterPicker.SelectedDate.HasValue)
        {
            DateTime startDate =
                StartDateFilterPicker.SelectedDate.Value.Date;
            filteredOrders = filteredOrders.Where(order =>
                order.OrderDate.Date >= startDate);
        }
        if (EndDateFilterPicker.SelectedDate.HasValue)
        {
            DateTime endDate =
                EndDateFilterPicker.SelectedDate.Value.Date;
            filteredOrders = filteredOrders.Where(order =>
                order.OrderDate.Date <= endDate);
        }
        List<Order> result = filteredOrders.ToList();
        OrdersDataGrid.ItemsSource = result;
        ResultsTextBlock.Text = $"Найдено: {result.Count}";
    }

    private static bool ContainsText(string? source, string searchText)
    {
        return !string.IsNullOrEmpty(source) &&
            source.Contains(searchText,
                StringComparison.OrdinalIgnoreCase);
    }

    private static void LoadOrderItemSummaries(SqlConnection connection,
        List<Order> orders)

```

Рис. 30. Листинг ProductsPage.cs

```

        if (orders.Count == 0)
        {
            return;
        }
        const string itemsQuery = @"
            SELECT oi.OrderId, p.Name, oi.Quantity
            FROM OrderItem oi
            INNER JOIN Product p ON oi.ProductId = p.Id
            ORDER BY oi.OrderId, oi.Id";
        using var command = new SqlCommand(itemsQuery, connection);
        using var adapter = new SqlDataAdapter(command);
        var itemsTable = new DataTable();
        adapter.Fill(itemsTable);
        var itemsByOrder = new Dictionary<int, List<string>>>();
        foreach (DataRow row in itemsTable.Rows)
        {
            int orderId = (int)row["OrderId"];
            string productName = row["Name"].ToString() ?? string.Empty;
            int quantity = (int)row["Quantity"];
            if (!itemsByOrder.TryGetValue(orderId, out List<string>?
items))
            {
                items = new List<string>();
                itemsByOrder[orderId] = items;
            }
            items.Add($"{productName} x{quantity}");
        }
        foreach (var order in orders)
        {
            order.ItemsSummary = itemsByOrder.TryGetValue(order.Id, out
List<string>? items)
                ? string.Join(" ", items)
                : "Нет позиций";
        }
    }
    private void NewOrderButton_Click(object sender, RoutedEventArgs e)
    {
        if (!AuthService.CanManageOrders())
        {
            MessageBox.Show("Создание заказа доступно только сотруднику
или администратору.", "Ограничение доступа", MessageBoxButton.OK,
MessageBoxImage.Information);
            return;
        }
        var dialog = new OrderDialog();
        if (dialog.ShowDialog() == true)
        {
            LoadOrders();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка открытия диалога: {ex.Message}",
"Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

```

Рис. 30. Листинг ProductsPage.cs

```

private void EditOrderButton_Click(object sender, RoutedEventArgs e)
{
    OpenSelectedOrder();
}

private void RefreshButton_Click(object sender, RoutedEventArgs e)
{
    LoadOrders();

    private void StatusFilterComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
    private void DateFilterPicker_SelectedDateChanged(object sender,
SelectionChangedEventArgs e)
    {
        if (StartDateFilterPicker.SelectedDate.HasValue &&
            EndDateFilterPicker.SelectedDate.HasValue &&
            StartDateFilterPicker.SelectedDate >
EndDateFilterPicker.SelectedDate)
        {
            if (ReferenceEquals(sender, StartDateFilterPicker))
            {
                EndDateFilterPicker.SelectedDate =
StartDateFilterPicker.SelectedDate;
            }
        }
        SearchTextBox.Clear();
        StatusFilterComboBox.SelectedIndex = 0;
        StartDateFilterPicker.SelectedDate = null;
        EndDateFilterPicker.SelectedDate = null;
        ApplyFilters();
    }
    private void OrdersDataGrid_SelectionChanged(object sender,
SelectionChangedEventArgs e)
    {
    }
    private void DeleteOrderButton_Click(object sender, RoutedEventArgs
e)
    {
        if (!AuthService.CanManageOrders())
        {
            MessageBox.Show("Удаление заказа доступно только сотруднику
или администратору.", "Ограничение доступа", MessageBoxButton.OK,
MessageBoxImage.Information);
            return;
        }
        try
        {
            if (OrdersDataGrid.SelectedItem is not Order order)
            {
                MessageBox.Show("Выберите заказ для удаления.",
"Информация", MessageBoxButton.OK, MessageBoxImage.Information);
                return;
            }
        }
    }

```

Рис. 31. Листинг OrdersPage.cs

```
private void OpenSelectedOrder()
{
    try
    {
        if (OrdersDataGrid.SelectedItem is not Order order)
        {
            MessageBox.Show("Выберите заказ.", "Информация",
                MessageBoxButton.OK, MessageBoxImage.Information);
            return;
        }

        bool isReadOnly = !AuthService.CanManageOrders();
        var dialog = new OrderDialog(order, isReadOnly);
        if (dialog.ShowDialog() == true)
        {
            LoadOrders();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка открытия заказа: {ex.Message}",
            "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Рис. 32. Листинг OrdersPage.cs